Comparison of Classical and Post-Quantum Cryptography on Two IoT Edge MCUs

By: Zain Ul Haq , Hochschule Bonn-Rhein-Sieg

1. Objective

This report measures what a TLS 1.2 public-key handshake costs on two microcontrollers, first with classical ECDH and ECDSA over P-256 and then with the NIST post-quantum replacements ML-KEM and ML-DSA. The measurements follow EEMBC methodology. SecureMark-TLS supplies the classical reference and the same 14-component structure is used for the post-quantum substitution, so the two can be put side by side.

The migration cost depends on the board and on the security level. At NIST Level 1 on the NXP MCXN947 the post-quantum handshake is 9.1% faster than the classical one, because the two signature verifications in the handshake become much cheaper. At Level 3 and Level 5 it is slower, by 50.9% and 125.9%. On the ESP32-S3 every level comes out faster, between 62% and 82%, which says more about the cost of its software ECC than about the cost of lattice arithmetic.

Everything below is measured. The code, the build settings and the command that reproduces each result are given with the experiment they belong to, and every step except the on-board runs executes on a PC from the stored data.

2. Method

2.1 Platforms

Property	NXP FRDM-MCXN947	ESP32-S3
Core	Cortex-M33, 150 MHz	Xtensa LX7, 240 MHz
Flash / SRAM	2 MB / 512 KB	8 MB / 512 KB
PQC implementation	PQClean	PQClean
Classical stack	mbedTLS 3.6.3	ESP-IDF mbedTLS

2.2 Measurement harness

Latency comes from hardware cycle counters, DWT->CYCCNT on the MCXN947 and the CCOUNT register on the ESP32-S3, both read through a small shim (dwt.h in the NXP projects, esp_timer on the ESP32 side) and converted to microseconds using each board's clock. Runs are taken with the debugger and the serial line detached, because either one attached shows up as jitter in the readings.

Peak stack use is measured rather than estimated: the task stack is painted with the byte 0xEE at boot and scanned afterwards for the deepest byte a run left untouched. Energy comes from a Nordic PPK2 at 10 kHz and 3.3 V in series with the board supply, the J24 rail on the MCXN947 and the 3V3 pin on the ESP32-S3, with the traces analysed by analyze_energy.py in codebase/projects/mlkem_bench/results/eembc_style. Captures are free-running so the measurement covers the computation and nothing else.

Sample counts are 100 repetitions per KEM operation and 2000 per signing call, with fresh randomness each time. The signing runs are what expose the latency tail in section 4.3.

One setting is not matched and it matters for section 4. The bare-metal ML-KEM build is -O0 while ML-DSA and the SecureMark primitives are -O2, so the post-quantum column carries more cost than it would at matched optimisation. The overhead reported there is an upper bound.

3. The PQC-extended SecureMark-TLS profile

The classical profile is the official EEMBC self-hosted SecureMark-TLS suite, 14 components that model a TLS 1.2 handshake with mbedTLS. The post-quantum variant keeps the same protocol structure and replaces the public-key primitives.

3.1 Primitive mapping

Classical phase (P-256)	Post-quantum equivalent
ECDH key exchange	ML-KEM keypair, encapsulation and decapsulation
ECDSA signatures	ML-DSA signing
ECDSA verification	ML-DSA verification

Classical 128-bit security (P-256) is compared against NIST Level 1 (ML-KEM-512 with ML-DSA-44), Level 3 (ML-KEM-768 with ML-DSA-65) and Level 5 (ML-KEM-1024 with ML-DSA-87).

3.2 Scoring

The EEMBC SecureMark score is 1000 divided by the weighted sum of the component processing times, so a larger score means less time per primitive. Each component is also checked against a reference CRC, and a score only compares to published figures if every check passes.

The score from this suite is the classical baseline for section 4, and it comes from two ports: `codebase/projects/securemark_tls_mcxn947` for the NXP board and `codebase/projects/securemark_tls_esp32s3` for the ESP32. The NXP port builds through the MCUXpresso SDK CMake flow with the Arm GNU toolchain, and its mbedTLS sources are compiled at -O2 through a per-file option, which that library needs for its inline MULADDC assembly on this core; the rest of the port stays at -O0. The ESP32 port builds with ESP-IDF v5.4 and CONFIG_COMPILER_OPTIMIZATION_PERF=y.

Both ports needed the same three fixes to run. mbedTLS wants an RNG for ECP point blinding on the ECDH call; without one it aborts with MBEDTLS_ERR_ECP_BAD_INPUT_DATA. EEMBC hands the peer key over as raw X||Y, so the SEC1 0x04 byte has to be prepended before parsing. And the signature must be DER encoded, since the CRC is computed over DER and not over raw r||s.

To reproduce, build and flash one of the ports and read its console. On the ESP32 side that is `bash codebase/projects/securemark_tls_esp32s3/scripts/build.sh build` followed by `bash codebase/projects/securemark_tls_esp32s3/scripts/build.sh -p COM17 flash`. A clean run prints 14 lines where crc equals expected_crc and then the score. The NXP port flashes with LinkServer, and its stored score is results/securemark_score.txt.

The MCXN947 scores 5472.08 marks and the ESP32-S3 scores 1895.06, both with 14 of 14 CRC checks passing.

4. Classical and post-quantum handshake comparison

The comparison uses an equivalent public-key phase on each side: one key exchange, one signature and two verifications. On the classical side that is ECDH, one ECDSA sign and two ECDSA verifies. The post-quantum side uses ML-KEM and ML-DSA in the same shape, so the two differ only in the primitives.

4.1 NXP FRDM-MCXN947 (150 MHz)

Suite	Latency (ms)	Delta
Classical, ECDH + ECDSA P-256	166.1	reference
PQC Level 1, ML-KEM-512 + ML-DSA-44	151.0	-9.1%
PQC Level 3, ML-KEM-768 + ML-DSA-65	250.7	+50.9%
PQC Level 5, ML-KEM-1024 + ML-DSA-87	375.2	+125.9%

The Level 1 result is the one that needs explaining. ML-DSA verification costs about a third of ECDSA verification, and the handshake performs two verifications, which more than covers the heavier ML-KEM key exchange and leaves the handshake 9.1% faster. Levels 3 and 5 grow the key exchange faster than the verification saving can offset.

4.2 ESP32-S3 (240 MHz)

Suite	Latency (ms)	Delta
Classical, ECDH + ECDSA P-256	495.2	reference
PQC Level 1, ML-KEM-512 + ML-DSA-44	88.8	-82.1%

PQC Level 3, ML-KEM-768 + ML-DSA-65	134.2	-72.9%
PQC Level 5, ML-KEM-1024 + ML-DSA-87	187.1	-62.2%

The ESP32-S3 is faster at every level after migration. The absolute saving is large, but it comes mostly from the classical side being expensive on this core, where ECC runs in software. It does not mean lattice arithmetic is cheap; the post-quantum absolute times are still the tens of milliseconds shown above.

4.3 The signing tail

Handshake budgets tend to be built from average times, and for ML-DSA that is unsafe. The signing median and worst case, over 2000 draws per level at 150 MHz, are:

Level	Median (ms)	Worst case (ms)	Worst / median
ML-DSA-44	43.7	262.9	6.0x
ML-DSA-65	75.7	611.1	8.1x
ML-DSA-87	99.6	475.7	4.8x

The worst case reaches six to eight times the median. The cause is rejection sampling inside the signing routine, which draws and discards candidates until one satisfies the norm bounds, so the tail belongs to the algorithm rather than to the board. Section 7 shows the same ratios on both cores. A scheduler that budgets the median will meet its deadline in testing and miss it in the field.

Both sides of the comparison are combined by two scripts that run on a PC from the stored per-operation times, so neither needs a board: `migration_compare.py` in the MCXN947 project for the first table and `cross_platform_compare.py` in the ESP32 project for the second. Their inputs are the SecureMark times in section 3, the ML-KEM times from `codebase/projects/mlkem_bench` built at -O0, and the ML-DSA times from `codebase/projects/mldsa_bench_nxp` built at -O2.

From the repo root, `python codebase/projects/securemark_tls_mcxn947/migration_compare.py` prints -9.1% and +50.9%, and `python codebase/projects/securemark_tls_esp32s3/cross_platform_compare.py` prints -82.1, -72.9 and -62.2. The tail numbers are stored per call as `mldsa*_sign_raw.csv` in mldsa_bench_nxp/results; to re-measure rather than reuse them, build that bench at the level you want with `cmake -DMLDSA_LEVEL=65 -DCMAKE_C_FLAGS=-O2`, flash, and read the percentiles from the new CSV.

5. Energy

5.1 Session resumption with DTLS Connection IDs

Running the full handshake on every wake-up is the dominant energy cost at high duty cycles. This section models a 24-hour deployment on a CR2032 cell (225 mAh) and compares a full handshake each wake-up against resuming with a DTLS Connection ID, which replaces the public-key phase with symmetric session resumption. The per-handshake energy fed into the model is measured; the model itself is arithmetic on that number and not a separate capture.

Board	Wake-ups / day	Full handshake (mJ)	With CID (mJ)	Saving
MCXN947	24	1340.9	1297.9	3.2%
MCXN947	288	1834.9	1298.2	29.2%
MCXN947	2880	6685.2	1301.5	80.5%
ESP32-S3	24	1735.1	1315.6	24.2%
ESP32-S3	288	6565.6	1330.7	79.7%
ESP32-S3	2880	53991.6	1478.7	97.3%

At the lowest rate the saving is small, 3.2% on the MCXN947. As the wake-up rate rises the full handshake becomes the entire budget and the saving grows to 97.3% on the ESP32-S3, which extends the cell from about 88 days to about 1740. The model is `dtls_cid_model.py` in the MCXN947 project and runs on a PC from the measured energy: `python codebase/projects/securemark_tls_mcxn947/dtls_cid_model.py` writes results/dtls_cid/daily_energy.csv with every wake-up rate for both boards.

5.2 Energy per handshake (ULPMark style)

For the ML-KEM-768 handshake the measured energy is 6.63 mJ with the PQClean reference code, which is 151 handshakes per joule. An M4F-optimised build of the same handshake measures 3.00 mJ, or 333 handshakes per joule. The optimised figure is a partial score: its decapsulation path faults on this core, so only keypair and encapsulation were measured (section 7 discusses the fault).

The 6.63 mJ figure comes from the free-running PPK2 captures analysed by `python codebase/projects/mlkem_bench/results/eembc_style/analyze_energy.py`, which reads the exported traces in the energy/ folder beside it and rewrites energy_summary.csv. The optimised comparison comes from `codebase/projects/pqm4_bench_mcxn947`, built with the Arm GNU toolchain at -O0 and with the linker stack region raised to 0x20000 because the optimised code needs more stack than the 32 KB default.

6. Joint scheduling of perception and cryptography

A single core has to run the perception loop and the handshake together. The loop used here has a 33.3 ms frame budget. The handshake is sliced into non-blocking pieces that execute in the slack between frames. The scheduler decides two things per frame: which ML-KEM security level to run and how deep to run the multi-exit perception network.

Policy	Total time (s)	Energy (mJ)	Deadline misses
Naive, always full depth	78.4	3382	70
Static early exit	69.6	3206	70
Adaptive early exit	67.9	3173	70
Learned joint	45.9	2079	0

The learned scheduler cuts the time and energy by roughly 40% and removes the missed deadlines, which the fixed policies all leave at 70.

The two decisions are not the same kind of problem. Security level is a choice among three discrete points with known costs, and a simple budget-matching rule picks as well as anything learned, so it stays a rule. The exit decision sits on a continuous confidence axis, and the learned isotonic risk curves fit operating points to the measured data that a fixed threshold cannot locate.

The scheduler is `joint_scheduler.py` in codebase/training_archive/training/conformal. It runs on a PC and takes the measured per-exit and per-security-level costs as fixed inputs, so `python codebase/training_archive/training/conformal/joint_scheduler.py` reproduces the table without hardware. The learned line on the test split reads 45920.7 ms, 2078.9 mJ and 0 misses.

7. Verification and compliance

Fixed workload. Policy runs use a 1230-frame sequence with one handshake step every ten frames, which is 123 steps or 41 full handshakes. The sequence is drawn from the frozen test split by `build_freerun_workload.py` in codebase/training_archive/training.

Sampling. Signing is measured over 2000 iterations per level with a distinct message and fresh randomness on each call, and key exchange over at least 100 iterations per operation.

Energy calibration. All energy figures come from PPK2 captures taken free-running at 10 kHz, so the debugger and serial activity do not contribute.

Cross-core consistency. One Zephyr build of ML-DSA, at -Os (Zephyr's default size optimisation, not the -O2 a comment in prj.conf suggests), was run on both boards, with CONFIG_THREAD_STACK_INFO enabled so the per-operation stack peaks are readable. The worst-to-median ratios are 7.1x, 6.0x and 5.9x for levels 44, 65 and 87, and they are identical on

the Cortex-M33 and the Xtensa, which rules out the silicon as the source of the tail. The project is `codebase/projects/pqc_bench_zephyr`, built with `west build -b frdm_mcxn947/mcxn947/cpu0 .` for the NXP board and `west build -b esp32s3_devkitc/esp32s3/procpu . -- -DEXTRA_DTC_OVERLAY_FILE=esp32s3_devkitc.overlay` for the ESP32 (the overlay routes the console to its USB serial port).

Numeric agreement between the board and the PC reference. The board's INT8 tensors are not bit-exact against the TFLite reference and differ by 1 to 4 LSB, growing slightly with depth:

Tensor	Max difference (LSB)
exit1 logits	1
block3 features	2
exit2 logits	3
block6 features	4
exit3 logits	3

The drift was traced to fixed-point accumulation inside the Neutron NPU. Driver and converter version skew was ruled out experimentally: once the driver was matched to the converter version the boot warning disappeared and the drift stayed byte-for-byte identical. The comparison tools are in codebase/projects/gtsrb_multiexit/tools/golden and the procedure is three steps: `python tools/golden/gen_golden.py` writes the reference and an embedded test header, the firmware is flashed and its GOLDEN lines captured over UART, then `python tools/golden/compare_golden.py <board_capture.txt>` diffs the two. Two things to know before running it. The reference generator needs a Python environment with TensorFlow, which the plain system interpreter does not have, and the comparison needs a capture from the board, since the board_capture.txt stored in the folder belongs to a different bench and will not compare. The figures in the table above are recorded in the deployment report alongside that capture procedure. A full audit of whether the drift ever changes the decided class across the whole validation set has not been done, so a decision-flip audit over the 1230 frames is open work rather than a result.

Implementation limit. The optimised M4F code path faults on this Cortex-M33 part, which is why section 5.2 covers only the phases that complete. The captured fault registers and the disassembly that locate the fault are in `codebase/projects/pqm4_bench_mcxn947/results/PQM4_M4F_M33_POSTMORTEM.md`.

8. References

EEMBC SecureMark-TLS, the classical TLS 1.2 benchmark used for the baseline.

EEMBC ULPMark, the energy-per-operation metric used to express section 5.2.

PQClean, the reference post-quantum implementations benchmarked here.

NIST FIPS 203 (ML-KEM) and FIPS 204 (ML-DSA), the standards for the post-quantum primitives.

IETF RFC 9146, DTLS Connection ID, the mechanism modelled in section 5.1.